

RELATÓRIO OPERACIONAL DE PRÉ-DECOLAGEM

Missão Aurora Siger - Validação de Parâmetros Críticos

1. Organização e Descrição da Telemetria

Antes de decidir qualquer coisa é preciso saber o que cada número enviado pela nave significa. A Aurora Siger transmite seis leituras por ciclo, divididas em dois grupos: cinco entram diretamente na decisão de lançamento e uma serve apenas para registro em log.

1.1 Parâmetros Monitorados

Parâmetro	Leitura atual	Unidade	Faixa segura	Papel na decisão
Temperatura interna	22,5	°C	18 a 26	Bloqueia o lançamento
Temperatura externa	-45,0	°C	—	Apenas registro
Integridade estrutural	1	0 ou 1	igual a 1	Bloqueia o lançamento
Nível de energia	85,0	%	acima de 80	Bloqueia o lançamento
Pressão dos tanques	42,0	atm	30 a 50	Bloqueia o lançamento
Status dos módulos críticos	OK	texto	igual a "OK"	Bloqueia o lançamento

1.2 Leitura de Cada Parâmetro

Temperatura interna e externa — a interna mede a cabine e os compartimentos pressurizados, e é a que interessa para a tripulação e para a eletrônica embarcada. A externa, de -45 °C, é registrada no log da missão, mas não impede a decolagem: o casco foi projetado para operar em ambiente extremo.

Integridade estrutural — indicador binário. O valor 1 significa casco íntegro e 0 significa falha detectada em algum ponto da fuselagem. A escolha por um valor binário é intencional: não existe casco parcialmente íntegro quando ele vai ser submetido à pressão do lançamento.

Nível de energia — percentual de carga das baterias principais. Serve tanto como critério de bloqueio quanto como entrada para o cálculo de autonomia da seção 4.

Pressão dos tanques — pressão interna dos tanques de propelente, em atm. Precisa ficar

dentro de uma janela: abaixo de 30 atm não há empuxo suficiente, e acima de 50 atm há risco de ruptura.

Status dos módulos críticos — resumo textual dos sistemas de navegação, suporte à vida e comunicação. Só o valor "OK" libera o lançamento; qualquer outro retorno é tratado como falha.

2. Algoritmo de Verificação

A decisão é tomada por uma regra conservadora: as cinco condições de bloqueio precisam ser atendidas ao mesmo tempo para que o lançamento seja liberado. Basta uma delas falhar para que o resultado seja DECOLAGEM ABORTADA. Não existe compensação entre os critérios — energia sobrando não cobre um casco comprometido.

2.1 Pseudocódigo

```
INÍCIO
  LER temperatura_interna, temperatura_externa, integridade_estrutural,
    nivel_energia, pressao_tanques, modulos_criticos

  REGISTRAR temperatura_externa no log da missão      // informativo

  SE (temperatura_interna ENTRE 18 E 26)
    E (integridade_estrutural = 1)
    E (nivel_energia > 80)
    E (pressao_tanques ENTRE 30 E 50)
    E (modulos_criticos = "OK")
  ENTÃO
    ESCREVER "PRONTO PARA DECOLAR"
  SENÃO
    ESCREVER "DECOLAGEM ABORTADA"
  FIM SE
FIM
```

2.2 Fluxograma

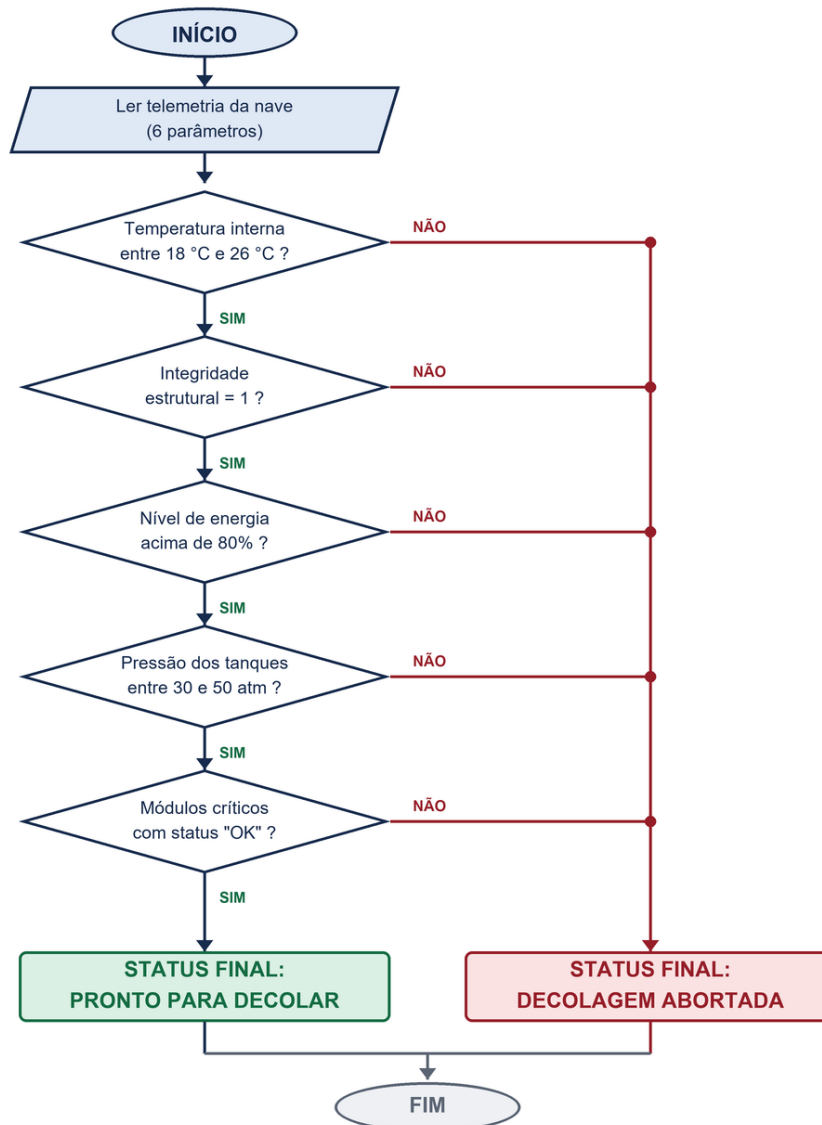


Figura 1 — Fluxo de decisão da verificação de pré-decolagem. Qualquer resposta NÃO desvia direto para o aborto.

3. Script em Python

O algoritmo foi implementado em Python puro, sem bibliotecas externas. O script lê a telemetria simulada, compara cada leitura com a faixa segura correspondente, imprime o resultado de cada checagem e conclui com o status do lançamento. Em seguida executa o cálculo de autonomia detalhado na seção 4.

3.1 Código-fonte

```

# -*- coding: utf-8 -*-
"""
Missão Aurora Siger - Verificação de pré-decolagem

```

Lê a telemetria da nave, compara cada leitura com as faixas seguras definidas para a missão e decide se o lançamento está liberado ou se deve ser abortado. Ao final, calcula a autonomia energética inicial.

Execução: `python verificacao_pre_decolagem.py`

```
"""
# -----
# 1. LEITURA DOS DADOS (telemetria simulada)
# -----
temperatura_interna = 22.5      # °C - cabine
temperatura_externa = -45.0    # °C - apenas registro, não impede o lançamento
integridade_estrutural = 1     # 1 = casco íntegro / 0 = falha detectada
nivel_energia = 85.0           # % da carga das baterias
pressao_tanques = 42.0         # atm
modulos_criticos = "OK"       # OK = navegação, suporte à vida e comunicação operantes

# -----
# 2. FAIXAS SEGURAS DEFINIDAS PARA A MISSÃO
# -----
TEMP_INTERNA_MIN = 18.0
TEMP_INTERNA_MAX = 26.0
ENERGIA_MINIMA = 80.0
PRESSAO_MIN = 30.0
PRESSAO_MAX = 50.0

# -----
# 3. EXECUÇÃO DAS VERIFICAÇÕES
# -----
print("--- INICIANDO VERIFICAÇÃO DE TELEMETRIA ---")
print()

checagens = [
    ("Temperatura interna",
     TEMP_INTERNA_MIN <= temperatura_interna <= TEMP_INTERNA_MAX,
     f"{temperatura_interna} °C (faixa: {TEMP_INTERNA_MIN} a {TEMP_INTERNA_MAX} °C)"),

    ("Integridade estrutural",
     integridade_estrutural == 1,
     f"{integridade_estrutural} (esperado: 1)"),

    ("Nível de energia",
     nivel_energia > ENERGIA_MINIMA,
     f"{nivel_energia}% (mínimo: acima de {ENERGIA_MINIMA}%)" ),

    ("Pressão dos tanques",
     PRESSAO_MIN <= pressao_tanques <= PRESSAO_MAX,
     f"{pressao_tanques} atm (faixa: {PRESSAO_MIN} a {PRESSAO_MAX} atm)"),

    ("Módulos críticos",
     modulos_criticos == "OK",
     f"{modulos_criticos} (esperado: OK)"),
]

for nome, aprovado, leitura in checagens:
    selo = "APROVADO" if aprovado else "REPROVADO"
    print(f"[{selo}] {nome:<24} -> {leitura}")

print(f"[REGISTRO] {'Temperatura externa':<24} -> {temperatura_externa} °C "
      "(informativo, não bloqueia o lançamento)")
print()
```

```

# -----
# 4. DECISÃO FINAL
# -----
if (18 <= temperatura_interna <= 26) and \
    (integridade_estrutural == 1) and \
    (nivel_energia > 80) and \
    (30 <= pressao_tanques <= 50) and \
    (modulos_criticos == "OK"):

    print("STATUS FINAL: PRONTO PARA DECOLAR")
else:
    print("STATUS FINAL: DECOLAGEM ABORTADA")

# -----
# 5. ANÁLISE ENERGÉTICA
# -----
CAPACIDADE_TOTAL = 100_000.0    # kWh
CONSUMO_DECOLAGEM = 30_000.0    # kWh
TAXA_PERDAS = 0.05              # 5% de dissipação térmica

carga_atual = CAPACIDADE_TOTAL * (nivel_energia / 100)
perdas = carga_atual * TAXA_PERDAS
consumo_total = CONSUMO_DECOLAGEM + perdas
autonomia_inicial = carga_atual - consumo_total

print()
print("--- ANÁLISE ENERGÉTICA ---")
print(f"Capacidade total .....: {CAPACIDADE_TOTAL:,.2f} kWh")
print(f"Carga atual ({nivel_energia}%) .....: {carga_atual:,.2f} kWh")
print(f"Perdas térmicas (5%) .....: {perdas:,.2f} kWh")
print(f"Consumo na decolagem .....: {CONSUMO_DECOLAGEM:,.2f} kWh")
print(f"Consumo total de lançamento ..: {consumo_total:,.2f} kWh")
print(f"AUTONOMIA INICIAL .....: {autonomia_inicial:,.2f} kWh")
print(f"Reserva sobre a carga atual ..: {autonomia_inicial / carga_atual * 100:,.2f}%")

```

3.2 Resultado da Execução

Saída obtida ao executar o script com a telemetria nominal da missão:

```

--- INICIANDO VERIFICAÇÃO DE TELEMETRIA ---

[APROVADO] Temperatura interna      -> 22.5 °C (faixa: 18.0 a 26.0 °C)
[APROVADO] Integridade estrutural   -> 1 (esperado: 1)
[APROVADO] Nível de energia         -> 85.0% (mínimo: acima de 80.0%)
[APROVADO] Pressão dos tanques      -> 42.0 atm (faixa: 30.0 a 50.0 atm)
[APROVADO] Módulos críticos         -> OK (esperado: OK)
[REGISTRO] Temperatura externa      -> -45.0 °C (informativo, não bloqueia o lançamento)

STATUS FINAL: PRONTO PARA DECOLAR

--- ANÁLISE ENERGÉTICA ---
Capacidade total .....: 100,000.00 kWh
Carga atual (85.0%) .....: 85,000.00 kWh
Perdas térmicas (5%) .....: 4,250.00 kWh
Consumo na decolagem .....: 30,000.00 kWh
Consumo total de lançamento ..: 34,250.00 kWh
AUTONOMIA INICIAL .....: 50,750.00 kWh
Reserva sobre a carga atual ..: 59.71%

```

3.3 Validação do Caminho de Aborto

Um algoritmo de segurança só tem valor se ele realmente barrar o que precisa barrar. Para confirmar isso, a mesma verificação foi executada com uma telemetria comprometida — casco com falha, bateria abaixo do mínimo, tanque sobrepressurizado, cabine acima de 26 °C e módulos críticos em falha. As cinco checagens reprovaram e o lançamento foi abortado, como esperado:

```
--- INICIANDO VERIFICAÇÃO DE TELEMETRIA ---  
  
[REPROVADO] Temperatura interna      -> 31.0 °C (faixa: 18.0 a 26.0 °C)  
[REPROVADO] Integridade estrutural    -> 0 (esperado: 1)  
[REPROVADO] Nível de energia          -> 62.0% (mínimo: acima de 80.0%)  
[REPROVADO] Pressão dos tanques       -> 57.0 atm (faixa: 30.0 a 50.0 atm)  
[REPROVADO] Módulos críticos          -> FALHA (esperado: OK)  
[REGISTRO] Temperatura externa        -> -45.0 °C (informativo, não bloqueia o lançamento)  
  
STATUS FINAL: DECOLAGEM ABORTADA
```

4. Análise Energética

Fora da atmosfera não existe tomada: toda a energia que a Aurora Siger vai consumir já está embarcada. Por isso o controle de consumo aqui é questão de sobrevivência, e não de economia. Mesmo as operações que parecem baratas — uma leitura de sensor, um recálculo de rota — gastam carga que só será reposta na primeira janela de recarga solar.

4.1 Parâmetros Definidos

- **Capacidade total:** 100.000 kWh
- **Carga atual:** 85% (85.000 kWh)
- **Consumo estimado na decolagem:** 30.000 kWh
- **Perdas energéticas:** 5% da energia total disponível (dissipação térmica decorrente de transmissão e conversão).

4.2 Memória de Cálculo

```
Carga Atual = 100.000 kWh * 0,85 = 85.000 kWh  
Perdas Térmicas = 85.000 kWh * 0,05 = 4.250 kWh  
Consumo Total de Lançamento = 30.000 kWh + 4.250 kWh = 34.250 kWh  
Autonomia Inicial (Pós-lançamento) = 85.000 kWh - 34.250 kWh =  
50.750 kWh
```

4.3 Conclusão Energética

A Aurora Siger inicia a viagem com 50.750 kWh disponíveis, o equivalente a 59,7% da carga com que entrou na contagem regressiva. É margem suficiente para manter suporte à vida, navegação e comunicação no período seguinte ao rompimento atmosférico.

5. Análise Assistida por IA

Durante a contagem regressiva ninguém consegue acompanhar manualmente o volume de telemetria que a nave envia. O papel da IA nesta etapa é ler esses dados antes do operador humano, apontar o que está fora do padrão e entregar um parecer de risco em que a equipe de solo possa se apoiar.

5.1 Classificação dos Dados (Telemetria)

Parâmetro	Leitura Atual	Classificação da IA
Temperatura Interna	22.5 °C	Rotina / Normal
Integridade Estrutural	1 (100%)	Rotina / Seguro
Níveis de Energia	85%	Rotina / Ideal
Pressão dos Tanques	42.0 atm	Rotina / Normal
Status Módulos Críticos	OK	Rotina / Operacional

5.2 Identificação de Possíveis Anomalias

Um modelo de detecção de anomalias aprende primeiro como é o comportamento normal dos sensores e só depois passa a sinalizar o que foge desse padrão. Na simulação atual não houve nada a sinalizar: as cinco leituras ficaram dentro das faixas e coerentes entre si. Se a integridade estrutural chegasse com valor 0, ou se a pressão dos tanques passasse de 50 atm, a leitura seria marcada como outlier e o sistema acusaria anomalia estrutural grave já na mesma varredura.

5.3 Análise Estatística e Empírica de Risco

- **Minimização de falhas e risco empírico:** o treinamento do modelo embarcado ajusta os parâmetros internos buscando o menor risco empírico possível, medido pela fórmula $\text{Risco empírico} = 1/n \sum L(y_i, f(x_i))$. Quem faz esse ajuste é um algoritmo de otimização — no caso, o gradiente descendente, que corrige os parâmetros de forma iterativa até a perda ficar pequena o bastante. É o ciclo comum a qualquer aprendizado supervisionado: parte-se dos dados rotulados e repete-se a sequência previsão, erro e correção.
- **Validação temporal dos sensores (evitando falsos positivos):** os dados da Aurora Siger vêm de sensores que medem continuamente, ou seja, formam uma série temporal. Embaralhar essas leituras para separar treino e teste, como se costuma fazer, provocaria vazamento temporal: o modelo veria eventos futuros durante o treinamento e passaria a acertar pelo motivo errado, mascarando falhas reais. Para evitar isso o sistema usa time series split, que faz cortes progressivos respeitando a ordem cronológica. Dessa forma o modelo é sempre treinado no passado e avaliado no futuro, de maneira repetida e estruturada.
- **Conclusão probabilística:** é essa validação mais rigorosa que permite confiar na

detecção de anomalias sobre dados históricos e estimar a probabilidade de falha antes da ignição dos motores, e não depois dela.

5.4 Sugestões de Risco

Risco avaliado: BAIXO. Todos os indicadores estão em estado nominal, sem indício de falha de hardware ou de vazamento energético. A recomendação do sistema é prosseguir com a contagem regressiva. Status sugerido: PRONTO PARA DECOLAR.

6. Reflexão Crítica

6.1 Ética e Responsabilidade

O algoritmo que autoriza ou aborta um lançamento carrega um peso diferente do de um software comum: o erro dele custa equipamento caro e, em missão tripulada, custa vidas. Isso cria duas obrigações práticas. A primeira é transparência — cada critério precisa estar registrado junto com a justificativa técnica do limite escolhido, e não apenas embutido no código. A segunda é auditabilidade: depois de um aborto, a equipe em Terra tem que conseguir apontar qual parâmetro reprovou e com qual leitura. Um sistema que decide sem explicar acaba transferindo para a máquina uma responsabilidade que continua sendo humana.

6.2 Impacto Social da Exploração Espacial

O efeito da exploração espacial não fica restrito ao que acontece fora da Terra. Boa parte do que se desenvolve para uma missão — materiais condutores, monitoramento por sensores, sistemas autônomos de decisão — termina voltando para o uso cotidiano, em hospitais, na agricultura e na previsão do tempo. Há também o outro lado da conta: programas espaciais são caros, financiados com dinheiro público ou por grandes grupos privados, e a pergunta sobre quem paga e quem colhe o benefício faz parte da discussão. Considerar os dois lados é mais honesto do que tratar o tema apenas como conquista tecnológica.

6.3 Sustentabilidade Tecnológica

A sustentabilidade tecnológica se apoia no conceito de Triple Bottom Line, o equilíbrio entre resultado econômico, equidade social e preservação ambiental. Numa nave isso aparece de forma bem concreta: cada watt perdido em dissipação térmica é autonomia a menos, e os 4.250 kWh calculados na seção 4 mostram o tamanho desse custo em uma única operação. Projetar hardware e software eficientes, desligar redundâncias quando elas não são necessárias e medir o consumo de cada subsistema são práticas alinhadas às diretrizes ESG — e que valem tanto a bordo quanto em um data center na Terra. O caso da Aurora Siger serve de lembrete de que ampliar a fronteira da computação e usar recursos com responsabilidade precisam caminhar juntos.

7. Repositório do Projeto

O código-fonte completo, o notebook executável e as instruções de execução estão no repositório público abaixo.

Link: <https://github.com/Joaomarcelloo-dev/missao-aurora-siger>

Estrutura do repositório:

```
missao-aurora-siger/  
├── missao_aurora_siger.ipynb      Notebook com todas as etapas  
├── verificacao_pre_decolagem.py   Script equivalente, para rodar no terminal  
├── README.md                     Explicação, prints e instruções de execução  
├── docs/  
│   ├── Relatorio_Operacional_Pre_Decolagem_Aurora_Siger.pdf  
│   ├── fluxograma_decisao.png  
│   └── prints/                   Capturas da execução
```